

Universal SKU Optimizer

Detailed Build Plan for One-Person Frontier Lab

Prepared for Shant Koutnouyan | 10-week vibe-code project plan

Executive summary

This document is a step-by-step plan to build an inventory optimization web app in 10 weeks using Claude Code as the primary engineering tool. The application accepts CSV or Excel uploads of any SKU panel, fits demand and lead-time distributions, runs forecasting models (classical + foundation), produces order-quantity and reorder-point recommendations under user-selected service levels, and exposes a natural language chat layer powered by Claude with tool use.

The plan is intentionally vibe-codeable: every feature is decomposed into a single-purpose Claude Code prompt, the tech stack avoids GPU compute, and the architecture is small enough to deploy on a free Vercel + Railway tier. Total expected API spend is under \$200 across the full build.

What you will have at the end of week 10

- **A live URL** where any user can upload a CSV or Excel file of SKU sales and get back a working dashboard with forecasts, reorder recommendations, and a chat interface.
- **A synthetic-data generator** that produces realistic SKU panels for demo scenarios when no user data is available.
- **Three pre-loaded demo datasets** (M5 Walmart subset, a coffee roaster scenario, an e-commerce scenario) so the demo never depends on a single CSV.
- **A natural language chat layer** with tool use: users ask 'which SKUs will stock out in 30 days?' and get answers grounded in their actual data.
- **A short demo video and one-pager** documenting the project for class submission and for showing prospective employers.

Contents

1. Project scope and deliverables
2. Architecture and tech stack
3. Data model and Excel/CSV ingestion
4. Forecasting layer
5. Inventory math layer
6. Natural language chat with tool use
7. Frontend specification
8. Synthetic and reference data
9. 10-week timeline with weekly milestones
10. Claude Code prompts library
11. Deployment, hosting, and cost
12. Testing, demo prep, and risks

1. Project scope and deliverables

1.1 Problem framing

Small and mid-market retailers, distributors, and DTC brands make ordering decisions in spreadsheets or on intuition. Enterprise tools like Blue Yonder and o9 cost six figures and require months of onboarding. Mid-tier tools like Inventory Planner and Cogsy run \$300-\$2,000/month and still require manual configuration per SKU. The opportunity is a vertical-agnostic, AI-native inventory tool that ingests messy data and returns defensible order recommendations within minutes.

1.2 In-scope features (v1, demo target)

- **Data ingestion:** CSV and Excel upload with auto-schema detection and a manual column-mapping fallback. Drag-and-drop interface, file size up to 50 MB, support for up to 100,000 SKU-period rows.
- **Synthetic data:** One-click sample dataset generation across three vertical templates (retail, coffee, e-commerce) with configurable noise, seasonality, and intermittency parameters.
- **Demand forecasting:** Per-SKU forecasts with prediction intervals using a hybrid of statistical baselines (statsforecast) and a foundation model (Chronos or TimeGPT) selected automatically based on series characteristics.
- **Lead time modeling:** Empirical lead-time distribution per supplier with bootstrap resampling for stochastic lead-time scenarios.
- **Inventory policies:** EOQ, (Q,R) continuous review, (s,S) periodic review, newsvendor for perishables, all with configurable service levels (90/95/99%) and holding/stockout cost inputs.
- **Single-SKU detail page:** Forecast chart, reorder recommendation, scenario sliders for lead time and service level.
- **Multi-SKU dashboard:** Sortable table of all SKUs with stockout risk, days-of-cover, recommended order quantity, and dead-inventory flags.
- **Natural language chat:** A Claude-powered chat that can answer questions about the loaded dataset using tool use (query SKUs, run forecasts, run scenarios).
- **Export:** Download recommendations as CSV or Excel; share dashboard via public link.

1.3 Out-of-scope (deliberately)

- Multi-echelon inventory optimization (network of warehouses).
- Real-time integrations with Shopify, Amazon, NetSuite, etc. (the file upload is the integration for v1).
- Multi-tenant authentication, billing, role-based access. Use a single password gate for the demo.
- Custom-trained ML models. We will use pretrained foundation models or out-of-the-box statistical methods only.
- Mobile app. Responsive web is sufficient.

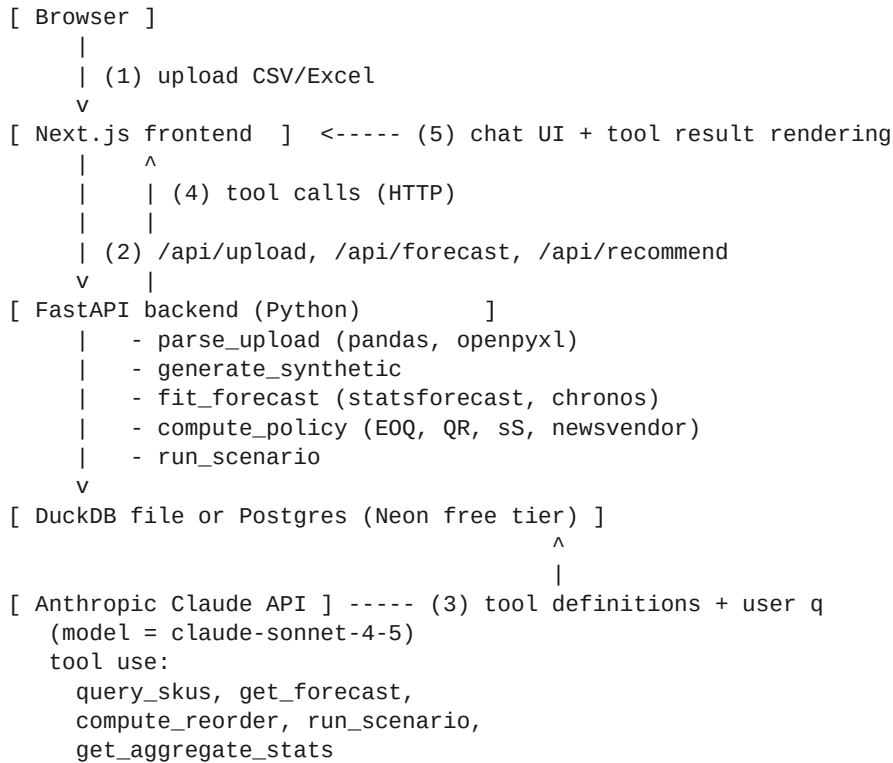
1.4 Success criteria for the class demo

- Live URL works without needing the presenter to be on-screen.
- Three different sample datasets demoable in under 90 seconds each.
- Chat layer answers at least 5 pre-tested user questions correctly with tool-grounded responses.
- At least one real third party (a friend, a Stanford classmate, a reachable SMB owner) has loaded their own data and gotten useful output.

2. Architecture and tech stack

2.1 High-level architecture

The app is split into three layers connected by HTTP: a Next.js frontend that handles the UI and file upload, a Python FastAPI backend that runs the forecasting and inventory math, and the Anthropic API which powers the natural language chat using tool use to call back into the FastAPI service.



2.2 Recommended stack with rationale

Layer	Choice	Why
Frontend framework	Next.js 14 (App Router)	First-class TypeScript, Vercel deploy in one click, server-side API routes for proxy logic, mature React ecosystem.
UI components	Tailwind CSS + shadcn/ui	Copy-paste primitives for tables, dialogs, sliders, charts. Designed to look good with zero design effort.
Charts	Recharts (React)	Built into shadcn dashboards. Sufficient for forecast lines, confidence bands, and bar/area charts.
File parsing (browser)	papaparse + SheetJS (xlsx)	Parse CSV and XLSX in the browser to give users instant feedback before uploading; full file then posted to backend for canonical processing.
Backend framework	FastAPI (Python 3.11+)	Type-safe, auto-generated OpenAPI docs, async-friendly, natural fit for pandas/numpy ecosystem where the math lives.
Database	DuckDB (file) for v1	Embedded, zero-ops, can be swapped for Postgres if needed. Lets you query a 100K-row SKU panel with SQL inside FastAPI.

Layer	Choice	Why
Forecasting	statsforecast + Chronos	statsforecast (Nixtla) for ETS, ARIMA, Croston. Chronos (Amazon, Hugging Face) for foundation-model zero-shot. Both run on CPU.
Inventory math	Custom Python module	EOQ, (Q,R), (s,S), newsvendor are short, well-defined functions. No need for a library.
LLM	Anthropic Claude API (claude-sonnet-4-5)	Best price/quality for tool use. Claude Code itself is also Sonnet-class so you stay in one mental model.
Auth (optional)	NextAuth or Clerk	Skip for v1. Add a single shared password env var if you want to gate the demo.
Hosting (frontend)	Vercel	Free tier is generous; Next.js native; preview URLs for every PR.
Hosting (backend)	Railway or Fly.io	\$5/mo plan handles a single FastAPI worker comfortably. Alternative: Modal (serverless Python with a generous free tier).
Storage (uploads)	S3 or Cloudflare R2	Cheap object storage for raw uploaded files. R2 has no egress fees, good fit for a class project.

2.3 Repository layout

```

inventory-optimizer/
  apps/
    web/                                # Next.js frontend
      app/
        page.tsx                       # landing
        upload/page.tsx                # file upload + column mapping
        dashboard/page.tsx            # SKU table + chat sidebar
        sku/[id]/page.tsx             # single-SKU detail
        api/                           # API routes (proxy to FastAPI)
      components/
        UploadDropzone.tsx
        ColumnMapper.tsx
        SkuTable.tsx
        ForecastChart.tsx
        ChatPanel.tsx
      lib/
        parseFile.ts
        api.ts
    api/                                # FastAPI backend
      main.py                          # entrypoint, routes
      ingestion.py                     # parse + validate uploads
      synthetic.py                     # synthetic data generator
      forecasting.py                   # statsforecast + chronos wrappers
      inventory.py                     # EOQ, QR, sS, newsvendor
      tools.py                         # JSON-schema tool definitions for Claude
      db.py                            # DuckDB helpers
  data/
    samples/                           # bundled sample CSVs
  scripts/
    seed_demo.py
  docs/
    CLAUDE.md                         # project context for Claude Code
    PROMPTS.md                        # the prompt library from this plan
  pyproject.toml
  package.json
  README.md

```

3. Data model and Excel/CSV ingestion

3.1 Canonical data schema

Internally the app normalizes everything into a long-format table with the following columns. All ingestion paths converge on this shape.

Column	Type	Required	Description
sku_id	string	yes	Unique identifier per SKU. Trimmed and uppercased on ingest.
date	date	yes	Period start date. Daily, weekly, or monthly granularity is supported; the backend infers periodicity.
demand	float	yes	Units sold or shipped in the period. Negative values are treated as returns and netted.
on_hand	float	no	Current on-hand inventory snapshot for the SKU. Optional; can be entered manually in the UI.
lead_time_days	float	no	Lead time observation for that order. If column is per-row, used to fit empirical distribution.
unit_cost	float	no	Cost per unit. Defaults to 1.0 if missing.
unit_price	float	no	Sale price per unit. Used for newsvendor and lost-sales costing.
supplier	string	no	Supplier name. Used to group lead-time observations.
category	string	no	Category or product family for grouped views.

3.2 Upload workflow

The upload flow is the most user-visible piece and gets the most polish budget. The user experience must feel as forgiving as Stripe's CSV import, not as brittle as enterprise ERP imports.

- **Step 1 - File selection.** Drag-and-drop or click. Accept .csv, .tsv, .xlsx, .xls. Show a progress bar while parsing.
- **Step 2 - Browser-side preview.** Use papaparse or SheetJS to read the first 50 rows and infer the schema in the browser. Show a 5-row preview with detected column types.
- **Step 3 - Column mapping.** Auto-detect mapping using header name fuzzy match (date, dt, day, period - sku, sku_id, item, product - demand, qty, units, sales). If confidence is low, show a column-mapping UI with dropdowns for each canonical field.
- **Step 4 - Validation.** Server-side validation runs on the full file. Common errors surfaced inline: missing required columns, dates that fail to parse, SKU-date duplicates, negative or non-numeric demand.
- **Step 5 - Persist.** Store the raw file in object storage, write the parsed canonical table to DuckDB. Return a dataset_id used by all subsequent endpoints.

3.3 Excel-specific considerations

- Use openpyxl on the backend to read .xlsx; xlrd is deprecated for .xls and you should warn the user to convert.
- Detect and skip blank rows above the header (Excel files often have title rows or merged header cells).
- Multi-sheet workbooks: if more than one sheet is present, prompt the user to select; default to the first sheet that has ≥ 50 rows and detected demand-like columns.
- Date columns from Excel arrive as datetime objects; do not assume strings. Time zones should be stripped.

4. Forecasting layer

4.1 Series characterization

Different SKUs need different models. Before forecasting, the backend characterizes each series and routes it to the appropriate method. The classifier is rule-based, transparent, and easy to override.

Pattern	Detection rule	Default method
Smooth	Coefficient of variation < 0.5; <= 30% zeros	ETS or AutoARIMA (statsforecast)
Seasonal	Detected seasonality at weekly or monthly lag (ACF peak)	Seasonal ETS or seasonal AutoARIMA
Intermittent	30%-70% zero-demand periods	Croston (or TSB variant)
Lumpy	> 70% zeros and high non-zero variance	Croston-SBA + lower confidence weight
Trending / new	< 13 observations or strong trend	Chronos zero-shot foundation model
Promo-driven	Optional promo flag column present	ML model (lightgbm) on lagged + calendar features

4.2 Foundation models for cold-start

For SKUs with limited history, foundation forecasting models trained on millions of public time series outperform anything you could build from scratch. The pipeline calls Chronos-Small (Hugging Face, free, runs on CPU in seconds) for these cases. Optionally swap in TimeGPT (Nixtla API, paid but trivial cost) if you want a faster API and a small accuracy bump.

4.3 Output format

Every forecast returns a uniform structure regardless of the underlying model:

```
{
  "sku_id": "ABC-123",
  "method": "ets" | "arima" | "croston" | "chronos" | "ensemble",
  "horizon_periods": 12,
  "frequency": "W",
  "point_forecast": [12.3, 14.1, 13.8, ...],
  "lower_80": [...], "upper_80": [...],
  "lower_95": [...], "upper_95": [...],
  "diagnostics": {
    "mape_backtest": 0.18,
    "bias_backtest": -0.03,
    "n_obs": 78,
    "characterization": "intermittent"
  }
}
```

4.4 Backtesting

For every forecast, the backend runs a rolling-origin backtest using the last N periods as a holdout. MAPE and bias are reported alongside the forward forecast so users see how trustworthy the recommendation is. This is also what the chat layer cites when asked 'how confident are you'.

5. Inventory math layer

5.1 Policies implemented

The math here is short and mostly textbook. Each policy is a small Python function with clear inputs and outputs, easy to test, easy to expose to the chat layer as a tool.

Policy	When to use	Key formula / approach
EOQ	Stable demand, deterministic lead time	$Q^* = \sqrt{2 \cdot D \cdot K / h}$, where D is annual demand, K is order cost, h is holding cost per unit per year.
(Q, R) continuous review	Stochastic demand, continuous monitoring	Q from EOQ; $R = \mu_{LTD} + z \cdot \sigma_{LTD}$ where μ_{LTD} and σ_{LTD} are mean and stdev of demand over lead time, z is the service-level z-score.
(s, S) periodic review	Periodic ordering, lumpy demand	s = reorder threshold; S = order-up-to level. Solved by simulation under fitted demand and lead-time distributions.
Newsvendor	Single-period perishable	$Q^* = F^{-1}\left(\frac{C_u}{C_u + C_o}\right)$ where C_u is underage cost (lost margin), C_o is overage cost (waste).
Base-stock	Make-to-stock with short cycle	Order up to $S = \mu_{LTD} + z \cdot \sigma_{LTD}$ every period.

5.2 Service levels and cost inputs

- **Cycle service level (alpha):** Probability of no stockout per cycle. Default 95%.
- **Fill rate (beta):** Fraction of demand met from stock. More natural for retail; expose both.
- **Holding cost rate:** Default 25% of unit cost per year. Allow per-SKU override.
- **Order cost (K):** Default \$50 per order. Allow per-supplier override.
- **Stockout cost:** Default = lost gross margin (price - cost). Allow override for B2B contracts where stockouts trigger penalty fees.

5.3 Scenario engine

Beyond the recommended policy, the user can run what-if scenarios. The engine takes a base case and a list of perturbations and re-runs the policy. Examples: 'what if lead time doubles', 'what if demand grows 20% YoY', 'what if I cut service level from 99% to 95%'. The chat layer exposes this as a tool so users can ask in plain English.

6. Natural language chat with tool use

6.1 Why tool use, not RAG

User questions about inventory require precise computation, not retrieval. 'Which SKUs will stock out in 30 days' is a query plus a forecast plus a policy run, not a vector lookup. Anthropic's tool use feature lets Claude decide which functions to call, with what arguments, and chain results into a final answer. The functions are the same FastAPI endpoints the UI uses.

6.2 Tool definitions exposed to Claude

Tool	Purpose
query_skus(filters, limit)	Return list of SKUs matching filters (category, supplier, demand range, days_of_cover range, etc.).
get_forecast(sku_id, horizon_periods)	Return forecast and confidence intervals for a SKU.
compute_reorder(sku_id, service_level, lead_time_days)	Run inventory policy and return recommended order quantity, reorder point, expected stockout probability.
run_scenario(sku_ids, perturbations)	What-if engine. Perturbations include lead_time_multiplier, demand_growth, service_level_target, holding_cost_rate.
get_aggregate_stats(group_by)	Portfolio-level metrics: total inventory value, dead stock, expected stockouts, working capital tied up.
get_sku_details(sku_id)	Full record for a SKU: history, distribution fit, current on-hand, supplier, recent forecasts.

6.3 System prompt template

You are an inventory analyst assistant for a small retailer.
You have access to tools that query the user's SKU dataset, run forecasts, and compute reorder recommendations. Always:

1. Call tools rather than guessing. If you need numbers, fetch them.
2. Show your reasoning briefly, then give a concrete recommendation.
3. Quantify uncertainty: cite backtest MAPE or confidence intervals when making forecasts.
4. If a question requires running a scenario, call run_scenario and summarize the comparison between base case and scenario.
5. Refuse to make recommendations on SKUs with fewer than 8 historical observations unless the user explicitly accepts low confidence.

The dataset has the following SKUs and metadata: {dataset_summary}.
Today's date is {today}.

6.4 Pre-tested demo questions

Have a list of 8-10 questions that you have manually verified work end-to-end. These are what you fall back on during the live demo.

- Which 5 SKUs are most likely to stock out in the next 30 days?
- What is my total inventory value and how much of it is dead stock?
- If lead times double for supplier X, how much extra safety stock do I need?
- Which SKUs should I drop if I want to cut inventory by 20%?

- Recommend an order for SKU ABC-123 with a 99% service level.
- Which categories have the worst forecast accuracy and why?
- What is my optimal reorder schedule for the next 90 days?
- Compare a 95% vs 99% service level on my top 10 SKUs - what is the cost?

7. Frontend specification

7.1 Pages

- **Landing (/):** One-line value prop, 'Try it now' CTA leading to upload, 'See sample data' button leading to dashboard with M5 subset preloaded.
- **Upload (/upload):** Dropzone, Excel/CSV preview, column mapping, validation results, 'Continue to dashboard' button.
- **Dashboard (/dashboard):** Two-column layout. Left: SKU table with sort, filter, search, KPIs at top. Right: collapsible chat panel.
- **SKU detail (/sku/[id]):** Full forecast chart with confidence bands, recommended policy card, scenario sliders, history table.
- **Settings (/settings):** Adjust service level defaults, holding cost rate, order cost. Per-supplier overrides.

7.2 Component checklist

- **UploadDropzone:** file selector, MIME validation, progress indicator, error surfacing.
- **ColumnMapper:** grid of canonical fields with dropdowns of detected file headers; live preview of mapped table.
- **SkuTable:** virtualized table (TanStack Table), 100k rows performant, sticky header, column visibility toggle.
- **ForecastChart:** Recharts area chart with point forecast and 80% / 95% confidence bands, hover tooltip with exact values.
- **ChatPanel:** message list, streaming token rendering, tool-call expandable cards (so the user sees what was looked up), scenario chips.
- **KpiCards:** total inventory value, expected stockouts (next 30d), dead stock value, average days of cover.
- **ScenarioSlider:** binds to one input (lead time multiplier, service level), debounced API call, inline diff vs base case.

7.3 Visual style

Default to shadcn/ui components and Tailwind defaults. Resist customization in v1. The signal of a serious tool is dense data, not custom theming. Use a single accent color for primary actions; default neutral palette for everything else.

8. Synthetic and reference data

8.1 Synthetic generator

A first-week deliverable. The generator creates realistic SKU panels with knobs for the characteristics that stress different parts of the system. Output format matches the canonical schema exactly so it flows through the full pipeline without special-casing.

- **Knobs:** `n_skus`, `n_periods`, `frequency` (D/W/M), `seasonality_strength`, `trend_slope`, `intermittency_rate`, `lead_time_mean`, `lead_time_cv`, `promo_frequency`, `supplier_count`.
- **Outputs:** a long-format CSV identical to a user upload; a metadata file describing the ground-truth distributions used; an evaluation harness that scores the app's forecasts against the known truth.
- **Templates:** three pre-baked configurations (retail-stable, coffee-perishable, ecommerce-lumpy) usable as one-click sample loads.

8.2 Public reference datasets

Dataset	Source	Use
M5 Forecasting (Walmart)	Kaggle (free, login required)	Hierarchical retail, 30K SKUs, gold standard for forecasting evaluation.
Rossmann Store Sales	Kaggle	1,100 stores, daily sales, includes promo flags.
Corporación Favorita	Kaggle	Ecuadorian grocery, real SKU panel.
UK Online Retail II	UCI ML repository	B2B e-commerce, ~1M transactions, lumpy patterns.
Olist	Kaggle	Brazilian e-commerce with real lead times and reviews.

9. 10-week timeline with weekly milestones

The plan front-loads the working pipeline so you have something demoable from week 2. Polish, NL features, and additional verticals layer on top. Every week ends with a specific deliverable that can be shown.

Week 1: Foundations + synthetic data

Tasks: Repo init (monorepo with Turborepo or workspaces). Vercel + Railway accounts and CI hooked up. FastAPI 'hello world' endpoint live. Next.js shell deployed to Vercel. Synthetic data generator with three templates committed.

End-of-week milestone: Synthetic CSV downloadable from a working live URL.

Week 2: Ingestion pipeline

Tasks: CSV/Excel upload route end-to-end: papaparse browser preview, FastAPI ingestion to DuckDB, schema auto-detection, column-mapping UI fallback, validation with friendly errors.

End-of-week milestone: User uploads any of three sample files and sees them parsed and stored.

Week 3: Inventory math module

Tasks: EOQ, (Q,R), (s,S), newsvendor, base-stock implemented as pure Python functions with unit tests. Service-level and cost defaults wired. Returns canonical recommendation object.

End-of-week milestone: POST /api/recommend with a sku_id returns a recommendation. Unit tests pass.

Week 4: Forecasting module

Tasks: statsforecast wrappers (ETS, ARIMA, Croston). Chronos integration via Hugging Face. Series characterization router. Backtest harness producing MAPE per SKU.

End-of-week milestone: POST /api/forecast returns point forecast + intervals + diagnostics for any SKU.

Week 5: Single-SKU detail page

Tasks: Frontend: SKU detail route with ForecastChart, recommendation card, scenario sliders bound to /api/recommend with perturbations. Visual polish pass.

End-of-week milestone: Click any SKU in the (still ugly) table, see the full detail page render.

Week 6: Multi-SKU dashboard

Tasks: Virtualized SKU table, KPI cards, sort/filter/search, dead-stock and stockout-risk flags. Performance pass to ensure 50k+ rows are smooth.

End-of-week milestone: Dashboard renders a 50k-SKU dataset without jank.

Week 7: Natural language chat layer

Tasks: Tool definitions for Claude (query_skus, get_forecast, compute_reorder, run_scenario, get_aggregate_stats, get_sku_details). System prompt + dataset summary injection. Streaming chat UI with tool-call cards.

End-of-week milestone: Chat answers all 8 pre-tested demo questions correctly.

Week 8: Polish and scenario engine

Tasks: Scenario engine surfaced as chips in chat ('What if lead time doubles?'). Export to CSV/Excel for recommendations. Settings page for service level and cost overrides. Empty states, error states, loading states.

End-of-week milestone: Demo flow runs end-to-end without manual workarounds.

Week 9: Outside testing + deployment hardening

Tasks: Send the live URL to 5-10 people (Stanford classmates, family, anyone with inventory data). Capture feedback, fix top 3 issues. Add basic logging and error tracking (Sentry free tier). Lock down env vars.

End-of-week milestone: Three external users have loaded their own data and given feedback.

Week 10: Demo prep

Tasks: Record a 4-minute demo video. Write a one-page write-up. Prepare a backup local environment in case Vercel is flaky on demo day. Rehearse the live walkthrough on three different sample datasets.

End-of-week milestone: Class demo delivered. Project artifact lives at a stable URL.

10. Claude Code prompts library

These are the prompts to paste into Claude Code, one per coding session. Each prompt assumes the previous prompts have been run, since Claude Code can read the existing repo. The first step is creating a CLAUDE.md file at the repo root - this gives Claude Code persistent context for every session.

10.1 Setup: CLAUDE.md (the persistent context file)

Create this file at /docs/CLAUDE.md. Claude Code automatically reads CLAUDE.md from the repo root and any docs/ subdirectory. It is the single most important file for keeping output consistent across sessions.

```
# Inventory Optimizer - Project Context

## What this app does
A web app that ingests CSV/Excel uploads of SKU sales data and returns
demand forecasts, reorder recommendations, and an LLM-powered chat
interface for inventory questions.

## Stack (do not change without explicit instruction)
- Frontend: Next.js 14 App Router, TypeScript, Tailwind, shadcn/ui,
  Recharts, TanStack Table.
- Backend: FastAPI (Python 3.11), pandas, statsforecast, chronos-forecasting,
  pydantic v2, openpyxl, duckdb.
- LLM: Anthropic API, claude-sonnet-4-5, tool use.
- Hosting: Vercel (web), Railway (api).

## Canonical data schema (long format, one row per SKU per period)
sku_id (str), date (date), demand (float), on_hand (float, optional),
lead_time_days (float, optional), unit_cost (float), unit_price (float),
supplier (str, optional), category (str, optional).

## Conventions
- Frontend code goes in apps/web. Backend in apps/api.
- All API routes are typed with pydantic models on the server and
  matching TypeScript types on the client (generated, not duplicated).
- Pure functions for math live in apps/api/inventory.py and have unit
  tests in apps/api/tests/.
- Never silently catch exceptions; always surface validation errors
  with field-level messages.
- Use server components by default in Next.js; mark client components
  explicitly.

## Out of scope for v1
Multi-tenant auth, real-time integrations (Shopify/Amazon), multi-echelon
optimization, custom ML training, mobile app.
```

10.2 Week 1: Initialize the monorepo

I want to scaffold a monorepo for an inventory optimization web app. Use a pnpm workspace at the root with two apps: apps/web (Next.js 14 App Router with TypeScript, Tailwind, and shadcn/ui initialized) and apps/api (FastAPI with pydantic v2, configured to run on uvicorn).

Set up:

- Root README.md describing the project (use the description from docs/CLAUDE.md).
- apps/web with a basic landing page that says "Inventory Optimizer" and has a button linking to /upload (the page can be empty for now).

- apps/api with a /health endpoint returning {"status":"ok"} and CORS configured to accept requests from localhost:3000.
- A scripts/dev.sh that starts both apps concurrently.
- A .env.example at the root with placeholders for ANTHROPIC_API_KEY, DATABASE_URL, and NEXT_PUBLIC_API_URL.

Do not deploy yet. Run both apps locally and verify the web app loads and can hit /health on the API.

10.3 Week 1: Synthetic data generator

In apps/api, create synthetic.py that exposes a function:

```
generate_synthetic(
    n_skus: int = 100,
    n_periods: int = 104,
    frequency: str = "W",
    seasonality_strength: float = 0.3,
    trend_slope: float = 0.0,
    intermittency_rate: float = 0.0,
    lead_time_mean_days: float = 14,
    lead_time_cv: float = 0.2,
    n_suppliers: int = 5,
    seed: int = 42,
) -> pd.DataFrame
```

It returns a long-format DataFrame with the canonical schema from docs/CLAUDE.md. Demand should be non-negative integers drawn from a mixture: zero-inflation per intermittency_rate, non-zero values from a negative binomial parameterized by per-SKU base level + seasonal + trend components. Lead times are per-supplier from a gamma distribution.

Also create three named templates that wrap this function:

- template_retail_stable
- template_coffee_perishable
- template_ecommerce_lumpy

Add a CLI (python -m apps.api.synthetic --template retail_stable --out data/samples/retail.csv) that writes a sample CSV. Add unit tests verifying the schema, no nulls in required fields, and that intermittency_rate is approximately respected.

10.4 Week 2: File upload and schema mapping

Build the upload pipeline.

Backend (apps/api):

- POST /datasets/upload: accepts a multipart file (csv, tsv, xlsx, xls). Use openpyxl for xlsx, pandas read_csv for csv/tsv. Returns a DatasetPreview with: detected_columns, suggested_mapping (best guess of canonical_field -> file_column using fuzzy header match), sample_rows (first 20), n_total_rows, dataset_id.
- POST /datasets/{dataset_id}/confirm: accepts the user-confirmed column mapping, validates the full file (required fields present, date parses, no SKU-date duplicates, demand numeric and ≥ 0 except returns), persists to DuckDB at data/datasets/{dataset_id}.duckdb, returns a DatasetSummary (n_skus, date_range, frequency, n_rows).

Frontend (apps/web):

- /upload page with shadcn Dropzone-style component, accepts the file

types above, shows a progress bar.

- After upload, call `/datasets/upload`, render a column-mapper component: a grid of canonical fields (`sku_id`, `date`, `demand`, `on_hand`, `lead_time_days`, `unit_cost`, `unit_price`, `supplier`, `category`) each with a Select dropdown populated from `detected_columns`. Pre-fill with `suggested_mapping`.
- Live preview pane shows the first 20 rows with the current mapping applied.
- "Confirm" button calls `/datasets/{id}/confirm` and routes to `/dashboard` on success.

Show validation errors inline. Use `react-query` for the API calls.

10.5 Week 3: Inventory math module

In `apps/api/inventory.py`, implement these pure functions with type hints and docstrings:

- `eqq(annual_demand: float, order_cost: float, holding_cost_per_unit: float) -> float`
- `qr_policy(mu_ltd: float, sigma_ltd: float, service_level: float, eqq_q: float) -> tuple[float, float]`
Returns (Q, R) where $R = \mu_{ltd} + z * \sigma_{ltd}$.
- `ss_policy_simulated(demand_samples: np.ndarray, leadtime_samples: np.ndarray, review_period: int, service_level: float, n_sims: int = 5000) -> tuple[float, float]`
Solves (s, S) by simulation under the empirical distributions.
- `newsvendor(underage_cost: float, overage_cost: float, demand_dist) -> float`
`demand_dist` is a `scipy` frozen distribution; return $F^{-1}(Cu/(Cu+Co))$.
- `base_stock(mu_ltd: float, sigma_ltd: float, service_level: float) -> float`

Add unit tests in `apps/api/tests/test_inventory.py` covering at least one numerical example per function with known answers (use textbook values).

Then expose `POST /skus/{sku_id}/recommend` that:

1. Loads the SKU's history from DuckDB.
2. Fits demand and lead-time distributions.
3. Selects a policy based on a simple rule: perishable category -> `newsvendor`; intermittent demand -> (s,S); else (Q,R).
4. Returns a Recommendation pydantic model with `policy_name`, `parameters`, `recommended_order_qty`, `reorder_point`, `expected_stockout_prob`, `expected_holding_cost`, `total_cost`.

10.6 Week 4: Forecasting module

In `apps/api/forecasting.py`:

1. Implement `characterize_series(series: pd.Series) -> str` returning one of {smooth, seasonal, intermittent, lumpy, trending_new}. Rules:
 - `n < 13` -> `trending_new`
 - `zero_pct > 0.7` -> `lumpy`
 - `zero_pct > 0.3` -> `intermittent`
 - `acf` has a significant peak at lag 7, 12, or 52 -> `seasonal`
 - else `smooth`.
2. Implement `forecast_classical(series, horizon, frequency)` using `statsforecast`: `AutoETS` for smooth, `AutoARIMA` for seasonal, `CrostonClassic` for intermittent and lumpy. Return point forecast plus 80 and 95 prediction intervals.
3. Implement `forecast_foundation(series, horizon)` using the `chronos-forecasting` library with model "amazon/chronos-t5-small". Run on CPU. Return same output shape.

4. Implement `forecast_sku(sku_id, horizon)` that loads the series from DuckDB, characterizes it, routes to classical or foundation, and returns a uniform Forecast object (point, lower_80, upper_80, lower_95, upper_95, method, diagnostics).
5. Implement `backtest(sku_id, n_holdout)` using rolling-origin evaluation; return MAPE, bias, n_obs.

Expose `POST /skus/{sku_id}/forecast?horizon=12` and add unit tests on synthetic series with known patterns to verify the characterization rules.

10.7 Week 5: Single-SKU page (frontend)

Build `/sku/[id]` in `apps/web`.

Layout: two-column on desktop, stacked on mobile.

Left column:

- SKU header (id, category, supplier).
- ForecastChart component (Recharts AreaChart): historical demand as a solid line, forecast point as dashed line, 80% interval as a darker band, 95% as a lighter band. Shared y-axis. Tooltip shows date, point, intervals.
- Diagnostics card: backtest MAPE, characterization, n_obs.

Right column:

- Recommendation card: policy name, recommended Q, R or s/S, expected stockout probability, expected total cost. "Re-run with overrides" button opens a dialog.
- Scenario sliders: service level (90-99%), lead-time multiplier (0.5x-3x), holding-cost rate (10-40%). Each change debounces by 400ms and calls `/skus/{id}/recommend` with perturbations; show a small diff badge vs base case (e.g., "+18 units, +\$420").
- History table (last 26 periods).

Use TanStack Query for fetching, suspense for loading. Generate the TypeScript types from the FastAPI OpenAPI schema using `openapi-typescript`.

10.8 Week 6: Multi-SKU dashboard

Build `/dashboard` in `apps/web`.

Top: KPI strip with cards for Total Inventory Value, Expected Stockouts (next 30d), Dead Stock Value, Average Days of Cover. Each card fetches its number from a new endpoint `/datasets/{id}/aggregate_stats`.

Main: SKU table using TanStack Table v8, virtualized with `@tanstack/react-virtual`. Columns: SKU, Category, Supplier, Days of Cover, Stockout Risk (color chip: low/med/high), Recommended Order Qty, Last Demand, Backtest MAPE. Sortable, filterable, with a free-text search across SKU and category. Click a row -> `/sku/[id]`.

Right sidebar: collapsible chat panel (placeholder for week 7).

Performance: must remain smooth with 50k rows. Test with a synthetic dataset of that size. Use Cell renderers that memoize.

10.9 Week 7: NL chat with tool use

Implement the natural language chat layer.

Backend (apps/api):

- POST /chat: accepts {dataset_id, messages} where messages is the conversation history. Server-side, call the Anthropic API with model claude-sonnet-4-5, with these tools defined in tools.py: query_skus, get_forecast, compute_reorder, run_scenario, get_aggregate_stats, get_sku_details. Each tool maps to a Python function that hits the existing service code.
- Implement a tool-use loop: call Anthropic, if response contains tool_use blocks, execute them, append tool_result blocks, call Anthropic again, until the response is final text. Stream the final text back to the client via Server-Sent Events.
- Inject a system prompt that includes a dataset summary: total SKUs, date range, top suppliers, frequency.

Frontend (apps/web):

- ChatPanel component. Message list with user/assistant bubbles. When the assistant calls tools, render a small expandable card per tool call showing tool name + arguments + result. Stream tokens for the final assistant message.
- "Quick questions" chips above the input with the 8 pre-tested demo questions. Clicking a chip submits it.

Add tests that verify each tool returns valid JSON for a known dataset state.

10.10 Week 8: Polish, scenarios, export

Polish pass. Implement:

- /api/datasets/{id}/export?format=csv|xlsx that returns a download containing one row per SKU with: sku_id, recommended_order_qty, reorder_point, expected_stockout_probability, total_cost, forecast_horizon_demand. Use openpyxl for xlsx.
- Settings page (/settings) with service level, holding cost rate, order cost defaults persisted per dataset.
- Empty states for the dashboard and SKU page (designed empty illustrations, not blank screens).
- Error boundaries on every route, with a friendly fallback and a "Reset" button.
- Loading skeletons for every async section (table, chart, KPI cards).
- Add a 'scenario chips' row inside ChatPanel: "What if lead times double?", "What if I cut service level to 90%?", "Top 5 stockout risks". Each chip submits a pre-filled question.

Run a Lighthouse pass and fix any P1 accessibility issues (labels on form fields, color contrast, keyboard nav).

10.11 Week 9: Hardening and outside testing

Production-readiness pass.

- Add Sentry (free tier) for error tracking on both web and api.
- Add structured logging on the API: every request logged with duration, status, dataset_id, sku_id when present.
- Rate-limit /chat to 30 requests per minute per IP using slowapi (FastAPI middleware).
- Add a single shared password gate (env var DEMO_PASSWORD) on the app: middleware in Next.js that requires a cookie set via /login.
- Run a full security review: ensure file uploads are size-limited (50 MB), MIME-validated, and stored under random UUIDs not user-

- supplied filenames. Ensure the chat layer cannot exfiltrate other users' data (scope every tool call to the dataset_id in the request).
- Write a 30-minute test plan and execute it: upload each of three sample files, navigate to a SKU, run a scenario, ask three questions in chat, export.

Then post the URL to 5-10 testers with a 5-question feedback form.

10.12 Week 10: Demo prep

Demo prep, no new features.

- Record a 4-minute Loom demo following this script: 1) drop in the M5 sample CSV (10s), 2) show the dashboard and KPIs (30s), 3) drill into one SKU and run a scenario (60s), 4) open chat and ask three of the pre-tested questions (90s), 5) export the recommendations as Excel (15s), 6) recap value prop (15s).
- Create a one-page write-up (PDF) with: problem, approach, screenshots, stack, results, link to live URL, link to GitHub.
- Set up a fallback: a local docker-compose that runs the full stack on the presenter's laptop, with the M5 dataset pre-loaded, in case the hosted version has issues during the live class demo.
- Rehearse the live walkthrough three times against different sample datasets.

11. Deployment, hosting, and cost

Item	Conservative monthly cost	Notes
Vercel (frontend)	\$0	Hobby tier sufficient for a class demo.
Railway or Fly.io (FastAPI)	\$5	Single small worker. Can also use Modal serverless and pay per request.
Cloudflare R2 or AWS S3	\$0-\$2	Object storage for raw uploaded files. R2 has no egress fees.
Domain (optional)	\$10-\$15 (one-time-ish)	Buy a .app or .ai domain for the demo.
Anthropic API	\$15-\$30 over 10 weeks	See section 11.2.
TimeGPT (optional)	\$0-\$10 over 10 weeks	Skip if you use Chronos. Cents per forecast otherwise.
Sentry	\$0	Developer tier free.

11.1 Total expected spend

Realistic case across 10 weeks: \$80 to \$120. Conservative budget: \$200. Worst case (heavy iteration, runaway scripts): \$400. Set up a hard spending limit on the Anthropic console at \$200 to remove tail risk.

11.2 Anthropic API cost mechanics

- **Model:** claude-sonnet-4-5 for chat (best price/quality with tool use).
- **Pricing reference:** roughly \$3/million input tokens, \$15/million output tokens (verify current pricing on console.anthropic.com).
- **Per chat turn:** ~5K-15K input tokens (system prompt + tools + dataset summary + tool results) and ~500-2K output tokens. Approximately \$0.05 per turn before caching.
- **Prompt caching:** cache the system prompt + tool definitions + dataset summary. Cached tokens are billed at 10% of normal input rate. This typically reduces per-turn cost by 50-70%.
- **Hard limits to set:** max_tokens of 1024 on the LLM responses; max 5 tool-use iterations per turn; max 30 turns per session.

11.3 CI/CD

- **Vercel:** connect the GitHub repo, automatic preview deploys on every PR.
- **Railway:** connect the same repo, configure a Dockerfile for the FastAPI app, automatic deploys on main.
- **GitHub Actions:** run pytest and tsc on every push. Fail the build on test or type errors.

12. Testing, demo prep, and risks

12.1 Testing strategy

- **Unit tests (Python):** Every function in `inventory.py` has at least one test with a known textbook answer. Forecasting characterization is tested on synthetic series with known patterns.
- **Integration tests:** One end-to-end test that uploads a small sample CSV, calls `/datasets/upload` + confirm, then `/skus/X/forecast`, then `/skus/X/recommend`, and asserts the response shape and key fields.
- **Frontend smoke test:** Playwright script that loads the landing page, navigates to `/upload`, uploads a fixed sample CSV, and verifies the dashboard renders with at least one row.
- **Manual demo dry-run:** Run the full demo script weekly starting from week 6. Capture timing.

12.2 Risks and mitigations

Risk	Mitigation
Scope creep across verticals.	Stay vertical-agnostic until v2. Resist building 'a coffee mode' until the universal engine is solid.
Forecasting library install issues (Chronos, statsforecast).	Pin versions in <code>pyproject.toml</code> . Use a Dockerfile from week 1 so local and Railway environments match.
LLM tool-use loops cost more than expected.	Hard cap on tool-use iterations per turn. Log token usage per request. Set a \$200 spending alert on Anthropic console.
Live demo dependency on Vercel or Railway being up.	Maintain a docker-compose local fallback with M5 dataset pre-loaded. Bring laptop with backup hotspot.
File upload edge cases (Excel quirks, weird encodings).	Test against the three reference datasets and at least one real-world Excel file from a friend or family business early in week 2.
Forecast quality looks bad on a user's real data.	Always show backtest MAPE alongside the forecast. The chat layer should refuse to recommend on series with < 8 observations.
Single-developer burnout.	Stick to the timeline. If a week slips, cut polish from week 8 first, then drop the foundation-model integration before cutting core features.

12.3 Final note

Two principles dominate this plan. First, the universal upload engine is the project; vertical templates are skins on top. Second, every week ends with something demoable, even if ugly. The moment you have a CSV in and a recommendation out, you can iterate forever; do not let the feature backlog block you from reaching that moment in week 3.

If anything in this plan feels too prescriptive, the right move is to override it explicitly in `CLAUDE.md` so future Claude Code sessions stay aligned with your decision.